Please describe how link works on smash bros- from a game design character controller persp[ective

From a **game design / character controller perspective**, Link in **Super Smash Bros.** is built as a *hybrid zoner–swordsman* with deliberate weight and commitment. I'll reference the modern interpretation from Super Smash Bros. Ultimate, since it represents the most refined version of his kit.

---

1. Core Design Pillar: "Deliberate Control"

Link is intentionally designed around:

- **Commitment over speed**
- **Stage control via projectiles**
- **Mid-range sword dominance**
- **Rewarding preparation and setup**

He is *not* a rushdown character. His controller feel communicates weight and intention.

From a systems standpoint, his design reinforces:

- Slower startup frames on many attacks
- High damage and knockback reward
- Projectile-based space conditioning
- Tool synergy (bomb setups especially)

---

2. Movement Controller Design

Ground Movement

- **Moderate run speed**
- Noticeable initial dash but not explosive
- Strong traction (he stops fairly reliably)

Design goal:

> Link should feel grounded and stable, not slippery or hyper-mobile.

This supports:

- Shield-based play
- Spacing with tilts
- Projectile conditioning

Air Movement

- Average fall speed
- Controlled but not floaty air drift

Importantly:

- His aerials are powerful but not ultra-fast.
- Short hop aerials are strong spacing tools.
- Landing aerials create pressure but require timing.

This makes him feel:

> "Calculated in the air," not acrobatic like lighter characters.

---

3. Normals: The Sword as a Disjointed Controller Tool

Link's sword gives him **disjointed hitboxes** (his hurtbox doesn't overlap the blade). From a design perspective:

Why this matters:

- Encourages mid-range spacing
- Rewards proper positioning
- Punishes reckless approaches

Key Controller Design Intent:

- **Forward tilt:** Spacing poke
- **Up tilt:** Anti-air coverage
- **Neutral air:** Flexible pressure tool

- **Back air:** High kill power

These moves:

- Have meaningful startup
- Are unsafe if spammed poorly
- Are strong if spaced correctly

The result:

> Player skill expression comes from range control, not speed mashing.

---

4. Projectiles: Layered Zoning System

Link's projectile kit is one of the most layered in the game:

Neutral B – Bow

- Linear projectile
- Chargeable
- Forces jumps or shields

Design role:

> Basic conditioning tool.

Side B – Boomerang

- Outgoing and returning hitbox
- Creates time-delayed pressure
- Sets up follow-ups

Design role:

> Tempo manipulator.

Down B – Remote Bomb (Ultimate version)

This is the most important modern design shift.

Unlike earlier versions, Link can:

- Spawn bomb
- Hold it
- Throw it
- Detonate remotely

From a controller systems perspective, this introduces:

- **State management** (bomb active vs inactive)
- **Traps**
- **Combo extensions**
- **Edgeguard pressure**
- **Recovery mixups**

The bomb turns Link into a *setup character*, not just a zoner.

This dramatically increases:

- Skill ceiling
- Creativity
- Technical depth

---

5. Recovery Design (Up B – Spin Attack)

Recovery is:

- Linear
- Vulnerable from distance
- Strong out-of-shield option

Design implication:

- Encourages stage control rather than deep offstage play
- Makes positioning matter
- Creates tension when recovering

However:
Bomb recovery tech allows advanced players to:

- Extend horizontal recovery
- Mix timing

Again reinforcing:

> Preparation and tool synergy are core to Link's design.

---

6. Risk / Reward Philosophy

Link's controller design follows this logic:

Action	Risk	Reward
Raw smash attack	High	Very high
Projectile spam	Low	Conditioning only
Bomb setups	Medium	High conversion
Properly spaced aerial	Medium	Strong advantage

He is not designed for low-risk aggression.
He is designed for:

> Space control → Read opponent → Convert hard

---

7. Shield & Defensive Identity

Link has:

- Strong out-of-shield Up B
- Decent grab
- Heavy weight (harder to kill)

This supports:

- Defensive mid-range play
- Punishing unsafe approaches

The character controller supports a:

> "Hold ground, don't rush" archetype.

---

8. Skill Floor vs Skill Ceiling

Low Level

- Projectile spam
- Basic sword spacing
- Simple kill confirms

High Level

- Bomb confirms
- Z-drop aerial extensions
- Frame traps
- Tech chase setups
- Recovery mixups

This makes Link:

- Approachable to beginners
- Deep for competitive players

Which is strong character design.

---

9. Comparison to Original Smash Identity

In earlier entries like Super Smash Bros. Melee, Link was slower and more rigid.

In Ultimate:

- Faster
- More fluid
- Bomb adds depth
- Less clunky aerial game

He evolved from:

> "Heavy zoner"
> to
> "Technical setup-based midrange controller."

---

10. Summary: Character Controller Identity

From a pure design lens, Link functions as:

Archetype:

- Mid-range sword zoner
- Setup-based trap character
- Defensive advantage holder

Controller Feel:

- Weighty
- Deliberate
- Rewarding preparation
- High payoff on reads

Core Fantasy:

> A tactical warrior who controls space with tools, not speed.

If you'd like, I can next:

- Break down Link frame data from a systems balancing perspective
- Compare him to another zoner like Samus
- Or analyze how bomb mechanics change neutral theory

Please talk about ti from a technical standpoint as if I was going to program it

Perfect — let's approach Link's **Super Smash Bros. controller** from a **programmer/game engine perspective**, focusing on how you might implement his mechanics. I'll frame it as if you're designing the character controller and state machine in a fighting game engine.

---

1. Core Character Controller Architecture

Link can be implemented as a **state-driven character controller**:

```
States:
- Idle
- Walk / Run
- Jump (Short hop / Full hop)
- Fall / Fast fall
- Dash
- Attack (Ground / Aerial)
- Special (Neutral/Side/Up/Down B)
- Grab / Throw
```

Plain text

Each state defines:

- Allowed inputs
- Movement parameters
- Collision rules
- Priority over other states

---

2. Movement System

Ground Movement

```
Variables:
- walkSpeed = 1.2 units/frame
- runSpeed = 2.5 units/frame
- acceleration = 0.15 units/frame²
- traction = 0.05 units/frame²

Mechanics:
- Character velocity integrates horizontal input per frame
```

Plain text

Air Movement

```
Variables:
- airSpeed = 1.5 units/frame
- airAccel = 0.1 units/frame²
- gravity = 0.15 units/frame² (terminalVelocity = 3.5 units/frame)
- fastFallMultiplier = 2.0

Mechanics:
- Air velocity updates with input
```

Plain text

---

3. Attack System

Each move can be represented as an **action with startup, active, and endlag frames**:

```
Attack = {
    hitbox: {x, y, width, height},
    damage: 12,
    knockback: 5,
    angle: 45 degrees,
    startup: 5 frames,
    active: 3 frames,
    endlag: 15 frames,
    cancelable: boolean,
```

Plain text

- Sword moves: **disjointed hitboxes**
  - Hitbox does not overlap character hurtbox
  - Easier to implement via offset transform relative to pivot
- Projectiles:
  - Spawn as separate objects in the world
  - Physics-driven (linear velocity for bow, arc/boomerang logic)
  - Collision with stage and opponents triggers damage events
  - Remote bomb requires **state tracking** (planted, held, detonated)

---

4. Bomb Mechanics (Down B)

Link's bomb introduces a **multi-state object**:

```
BombState = enum {Idle, Held, Thrown, Planted, Exploding}

Bomb = {
    position: Vector3,
    velocity: Vector3,
    timer: int,
    owner: Link,
    damage: 20,
    knockback: 8,
```

Plain text

Logic:

1. On **spawn**, bomb is attached to character (`Held`)
2. If player releases input → `Thrown` with initial velocity
3. On ground collision → `Planted`
4. Player input triggers → `Exploding` after delay
5. Explosion applies AoE damage + knockback

- Must track **ownership** for combo logic (prevent self-damage issues)
- Integrates with attack hit detection system

---

5. Projectile Mechanics (Bow/Boemerang)

Bow (Neutral B)

- Projectile moves in a straight line
- Can be **charged** (chargeTime affects speed/damage)
- Collision detection with enemies triggers damage event

Boomerang (Side B)

- Outgoing trajectory followed by returning trajectory
- Returns to player if not intercepted
- Implement via **state machine per projectile**

Plain text

---

6. Recovery (Up B – Spin Attack)

- Movement vector defined per frame
- Can include **invincibility frames**
- Interruptible by grab or shielded hit
- Needs a **priority system** to override normal physics (e.g., wall collision, edge cling)

---

7. Input Handling

Link relies on **intentional input mapping**:

- Standard movement: analog stick → movement vector
- Button mapping:
  - A = standard attack
  - B = special attack
  - Up + B = recovery
  - Shield / Grab handled separately
- Combos / aerial sequences detected via **input queues** or **frame windows**

---

8. Damage & Knockback System

- Damage accumulates as a scalar per character
- Knockback vector = baseKnockback + scalingFactor × damage
- Launch angle determines trajectory (affects recovery and edgeguard logic)
- Must integrate with physics engine to simulate arcs and landing hits

---

9. State Priorities & Interrupts

- **Active attacks** have higher priority than movement
- **Shielding** can cancel ground normals

- **Specials** override neutral moves but may be interrupted if hit
- **Hitstun** always overrides current action → ensures proper physics response

Plain text

---

10. Physics Integration

Key physics considerations:

- Collision boxes: separate **hurtbox** vs **hitbox**
- Gravity and momentum affect all movement-based states
- Fast fall / fall speed multiplier allows player skill expression
- Edge collision for ledge grabbing and recovery

TL;DR Programmer's Mindset

1. **State machine-driven character controller**
2. **Weighted movement** with traction & gravity
3. **Hitboxes & attack frames** as modular objects
4. **Projectiles / bombs** as independent physics objects with state
5. **Recovery & specials** override base physics but must blend smoothly
6. **Priority system** to resolve conflicting actions
7. **Input queue / timing system** to allow skill expression

If you want, I can **draw a full technical diagram showing Link's states, transitions, and projectile interactions**, so it reads like a programmer's reference for building a Smash-like character. This would essentially be a blueprint for implementing him from scratch. Do you want me to do that next?

Yes, please a full full technical diagram.

Perfect! I've mapped out a **full technical diagram for Link's character controller** as if you were implementing him in a Smash-like engine. I'll describe it hierarchically and show all **states, transitions, and interactions with projectiles and bombs**, as a programmer-focused blueprint.

Link Character Controller Diagram (Technical Blueprint)

```
Link_Controller
│
├── Inputs
│   ├── Movement: Analog stick → horizontal/vertical vectors
│   ├── Attack: A button → normal attacks
│   ├── Special: B button → special attacks
│   ├── Jump: Up or X/Y → short hop / full hop
│   ├── Shield: L/R → shield / dodge
│   └── Grab: Z → grab / throw
│
├── States (State Machine)
│   │
│   ├── Ground States
│   │   ├── Idle
│   │   ├── Walk
│   │   ├── Run
│   │   ├── Dash
│   │   ├── Crouch
│   │   └── Shield
│   │
│   ├── Air States
│   │   ├── Jump (short / full hop)
│   │   ├── Fall / Fast Fall
│   │   ├── Aerial Attack (Nair, Fair, Bair, Uair, Dair)
│   │   └── Recovery (Up B)
│   │
│   ├── Attack States
│   │   ├── Ground Attack (Tilts, Smashes)
│   │   ├── Aerial Attack (Short hop Nair, etc.)
```

```
│   │   └── Grab / Throw
│   │
│   ├── Special States
│   │   ├── Neutral B: Bow
│   │   ├── Side B: Boomerang
│   │   ├── Up B: Spin Attack (Recovery)
│   │   └── Down B: Bomb
│   │
│   └── Hit / Knockback
│       ├── Hitstun
│       ├── Launch
│       └── Death / Respawn
│
├── Movement System
│   ├── Ground Physics
│   │   ├── walkSpeed / runSpeed / acceleration
│   │   ├── traction / friction
│   │   └── collision detection with stage
│   └── Air Physics
│       ├── airSpeed / airAccel
│       ├── gravity / terminalVelocity
│       └── fastFallMultiplier
│
├── Attack System
│   ├── Normal Attacks
│   │   ├── hitbox: offset from pivot (disjointed)
│   │   ├── startup / active / endlag
│   │   └── cancelable / interruptible flags
│   ├── Smash Attacks
│   └── Aerial Attacks
│
├── Special Objects
│   │
│   ├── Projectiles (Bow & Boomerang)
│   │   ├── Bow
│   │   │   ├── velocity, chargeTime, lifetime
│   │   │   └── collision → damage events
│   │   └── Boomerang
│   │       ├── StateMachine: Outgoing → Returning → Expired
│   │       └── collision logic
│   │
│   └── Bomb
│       ├── BombState: Idle → Held → Thrown → Planted → Exploding
│       ├── velocity / timer / owner reference
│       └── explosion → AoE damage + knockback
│
├── Collision & Hit Detection
│   ├── Hurtbox (character body)
│   ├── Hitboxes (attacks / projectiles)
```

```
│   └── Interaction logic (damage, knockback, hitstun)
│
├── Recovery / Up B
│   ├── Override movement vector
│   ├── Invincibility frames
│   └── Edge interaction (ledge grab)
│
├── Priority & Interrupt System
│   ├── Hitstun > Specials > Attacks > Movement
│   └── Input-buffered transitions
│
└── AI / Player Input Queue
    ├── Frame-based input handling
```

Code

Key Notes for Implementation

1. **State Machine Driven**
   - Every action is a **state**, transitions triggered by inputs, collisions, or frame events.
2. **Attack & Projectile Abstraction**
   - Attacks and projectiles are modular objects with `hitbox`, `damage`, `knockback`, and `lifetime`.
3. **Bomb as Multi-State Object**
   - Critical for setup/tech depth. Must track `owner`, `state`, and timer for detonation.
4. **Priority System**
   - Resolves conflicting states like movement + attack + hitstun.
5. **Physics Integration**

ChatGPT Log in Sign up for free

   - Air control uses acceleration and fast-fall multipliers.
6. **Frame-Based Input Buffer**
   - Enables responsive short hops, action buffering, and tech combos.

If you want, I can **draw this as a visual diagram with arrows showing transitions between all states** — it would look like a **full software engineering state diagram** ready for coding in Unity, Unreal, or a custom engine.

Do you want me to create the visual diagram next?

Please now describe how it works and what changes when the character picks up a weapon in the game. Eg, a pokeball, a wand, sword, etc